



PS를 위한 문제 해결사의 지침서

코드를 넘어 비즈니스 가치를 창출하는 17가지 질문

Based on Debbie Widjaja's Problem Solver's Playbook

오늘의 학습 여정: 3단계 프레임워크

문제 해결사(Problem Solver)로 진화하기 위한 로드맵

STEP 01

문제 해결의 가치 평가

"이것이 정말 해결할 가치가 있는 문제인가?"

- 증상과 근본 원인 구분하기
- 고객 및 비즈니스 영향력 측정
- 기회비용과 아무것도 안 할 시의 비용

STEP 02

중요한 문제 정의

"더 넓은 시야로 문제를 바라보고 있는가?"

- JTBD: 고객이 진짜 하려는 일
- 경쟁 우위와 네트워크 효과 구축
- 미래 시나리오와 제품의 지속성

STEP 03

최적의 해결책 찾기

"가장 효율적이고 실행 가능한 방법은 무엇인가?"

- 실현 가능성(Feasibility) 검토
- 한계효용 체감과 MVP 전략
- 레드팀: 비판적 시각으로 검증

결과물: 위임과 성장을 통해 PO/PS 개인의 생산성을 넘어 조직 전체의 가치를 극대화하는 법을 배웁니다.

우리는 왜 '문제 해결' 그 자체에 함몰될까요?

문제 해결사(Problem Solver)로 진화하기 위한 로드맵

TRAP 01: HOW OVER WHY



"어떻게 해결할까?"라는 본능

비즈니스 가치보다 기술적 구현 가능성에 매몰되는 현상

TRAP 02: THE SHINY OBJECT SYNDROME



'지루함'이라는 기술적 함정

해결 과정 중 마주치는 새로운 기술적 호기심에 본질을 놓침

PS는 기능을 만드는 사람이 아니라, '가치를 극대화'하는 사람입니다.

- 모든 문제를 혼자 해결하려는 'Hero Culture'는 자원 확보의 실패와 번아웃을 초래합니다.
- 문제 해결 그 자체의 즐거움보다, 그 해결이 가져올 '고객의 가치'를 우선순위에 두어야 합니다.

패러다임의 전환

'코드 작성'에서 '가치 극대화'로



"PS의 역할은 코딩이 아니라, 비즈니스 가치를 전달하는 것입니다."

문제나 해결책 그 자체가 아닌, 오직 그것이 만들어낼 '가치'에만 집착하십시오.

제1장: 문제 해결의 가치 평가하기

'코드 작성'에서 '가치 극대화'로



6단계의 문제 검증 프로세스

- #1 증상 vs 근본 원인 파악
- #2 고객에게 미치는 영향력(Reach/Intensity)
- #4 비즈니스 이점과 수익성
- #4 장기 전략 및 비전과의 정렬
- #5 기회비용 (무엇을 포기해야 하는가?)
- #6 '아무것도 하지 않았을 때'의 비용

"PS에게 가장 비싼 자원은 '시간'입니다. 연애 대상을 고르듯 신중하게 문제를 선택하십시오."

질문 #1: 증상에 속지 마세요, 근본 원인을 찾으세요

단순한 UI 수정보다 '해당 기능이 왜 필요한가'라는 본질적인 질문이 우선입니다.

SURFACE: SYMPTOMS (증상)

가입 양식 전환율 저조

(단순한 사용성 문제로 오해)

UNDERWATER: ROOT CAUSE (근본 원인)

WHY 1. 사용자가 가입 단계에서 이탈합니다.

WHY 2. 양식 작성이 번거롭다고 느낍니다.

WHY 3. 가입 후에 얻을 혜택이 불분명합니다.

WHY 4. 제품이 제공하는 핵심 가치를 이해하지 못했습니다.

ROOT: 기능 자체가 가치를 주지 못함

경계하세요: 확증 편향 (Confirmation Bias)

"내가 만든 코드는 완벽해, 단지 UI가 문제일 뿐이야"라고 믿고 싶어 하지는 않나요? 데이터를 통해 가설이 틀릴 수 있음을 인정하는 것이 Problem Solver의 첫걸음입니다.

질문 #02: 영향력을 정량화하는 공식

단순한 추측이 아닌 데이터로 증명하는 문제의 우선순위

THE IMPACT FORMULA

$$\text{Impact} = \text{Reach} \times \text{Intensity} \times \text{Segment}$$

Reach (도달 범위)

- 이 문제가 영향을 미치는 고객 수
- 실제 로그나 DB 데이터를 기반으로 산출하는 객관적인 지표

Intensity (강도)

- 사용자가 느끼는 고통의 수준
- 1~10점 척도 또는 '필수' vs '선택'
- 사용자 인터뷰 및 설문조사 기반

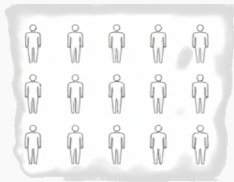
Segment (세그먼트)

- 대상 고객의 비즈니스 가치
- 유료 결제 유저, 핵심 타겟 그룹 등 비즈니스 임팩트에 따른 가중치

"PS의 논리적 사고를 영향력 계산에 적용하세요. 느낌이 아닌 수치로 우선순위를 결정할 때 팀을 더 잘 설득할 수 있습니다."

영향력(Impact)의 3가지 핵심 지표

데이터와 정성적 평가를 결합하여 문제의 크기를 정량화합니다.



01. 도달 범위 (Reach)

"얼마나 많은 사용자가 겪는가?"

- 실제 영향을 받는 고객 수
- 객관적이고 측정 가능한 수치
- 모수(Population)의 크기 확인



02. 강도 (Intensity)

"고통의 깊이가 어느 정도인가?"

- 1~10점 척도로 고통 지수 산정
- 사용자 조사를 통한 정성적 파악
- '있으면 좋은 것' vs '필수적인 것'



03. 세그먼트 (Segment)

"전략적으로 얼마나 중요한가?"

- 비즈니스 기여도에 따른 가중치
- VIP 고객 vs 일반 고객 구분
- 타겟 그룹별 배율(Multiplier) 적용

$$\text{IMPACT} = \text{REACH} \times \text{INTENSITY} \times \text{USER SEGMENT}$$

질문 #3 & #4: 비즈니스 가치와 전략적 정렬

PS가 코드를 넘어 비즈니스 임팩트를 고려해야 하는 이유

QUESTION #03

이 문제를 해결하면 회사에 어떤 이득이 있나요?



- 고객의 니즈가 곧 회사의 수익은 아닙니다.
- 해결책이 비즈니스 수익성(ROI)을 보장하는지 검토하세요.
- 단기적 이득보다 장기적 신뢰 하락의 비용을 계산하세요.

QUESTION #04

제품의 장기적인 비전 및 전략과 일치하나요?



- 수익성이 있어도 비전과 다르면 거절해야 합니다.
- "전략의 본질은 무엇을 하지 않을지 선택하는 것"
- 웹 대신 앱에 집중하기로 했다면, 웹 수정은 '아니오'.

훌륭한 PS는 '구현 가능한 기능'이 아니라 '비즈니스를 성장시키는 문제'를 푼다.

질문 #5 & #6: 기회비용과 방치의 비용

자원의 한계 속에서 무엇을 해결하고 무엇을 방치할 것인가

#5: "이 작업을 위해 어떤 우선순위를 낮춰야 할까요?"

PS의 시간은 유한합니다. 특정 기능을 개발하기로 결정했다면, 포기해야 하는 다른 가치가 무엇인지 계산하세요.

#6: "아무것도 하지 않으면 어떻게 될까요?" (방치의 비용)



화재 (Fire)

지금 해결하거나, 나중에 재건하거나

즉각적인 조치가 필요한 크리티컬한 버그나 장애



새는 지붕 (Leaking Roof)

방치하면 서서히 악화됨

성능 저하, 기술 부채, 지속적인 이탈 유발 요소



물웅덩이 (Puddle)

성가시지만 큰 해가 없음

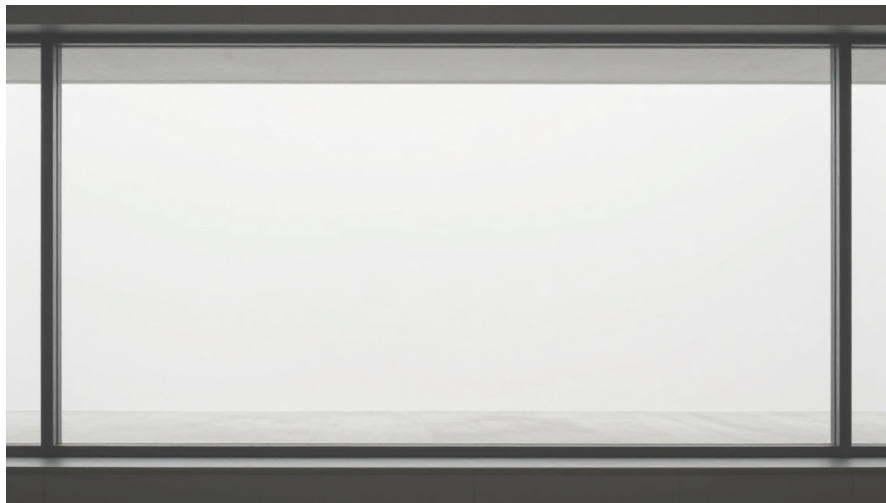
사소한 UI 어긋남, 비주류 케이스의 불편함

"모든 것을 고치는 것이 최고의 엔지니어링은 아닙니다."

- 훌륭한 문제 해결사는 방치해도 안전한 '물웅덩이'를 식별하고, 한정된 에너지를 '화재' 진압에 집중합니다.
- 완벽주의를 버리고 가치 극대화를 선택하세요. 때로는 '하지 않는 것'이 가장 효율적인 솔루션입니다.

제2장: 중요한 문제 찾기 (시야 넓히기)

특정 기능의 개선을 넘어, 비즈니스의 지평을 넓히는 사고방식



10%

기능 개선 (Junior)

주어진 문제의 효율화
사용성 및 UI 다듬기
코드 최적화에 집중

10X

기회 포착 (Senior)

본질적인 니즈의 재정의
새로운 시장 가치 창출
비즈니스 임팩트 설계

"나무가 아닌 숲을 볼 때, 해결해야 할 '진짜 문제'가 보입니다."

- 특정 기능에 매몰되지 않는 여유가 더 넓은 시야를 만듭니다.
- 고객의 본질적인 니즈와 시장의 변화를 읽는 4가지 핵심 질문을 시작합니다.

질문 #7: 고객은 왜 우리 제품을 '고용'할까요?

JTBD(Jobs-to-be-Done)로 보는 문제의 본질



우리 제품 (수단)

THE JOB



고객의 최종 목적 (가치)

"기능이 아닌 '목적'을 설계하세요"

- 고객은 단순히 도구를 쓰는 게 아니라, 삶의 특정 변화를 위해 제품을 고용합니다.
- PS의 시선이 사용성(UX)에만 갇히면 진짜 경쟁자를 놓칠 수 있습니다.

비행기의 경쟁자는 다른 항공사가 아닙니다

- 비행기를 타는 목적이 '출장지에서의 미팅'이라면, 경쟁자는 Zoom입니다.
- '이동'이라는 기능적 해결보다 '연결'이라는 본질적 니즈에 집중하세요.

질문 #8: 성을 지키는 깊은 물웅덩이, '해자' 구축하기

단순 기능 구현을 넘어, 경쟁자가 쉽게 넘볼 수 없는 기술적·심리적 진입 장벽 설계

네트워크 효과 (Network Effects)

사용자가 많아질수록 각 개인에게 제공되는 가치가 커지는 구조

높은 전환 비용 (Switching Costs)

기술적 종속성이나 데이터 마이그레이션의 어려움을 통한 유지

기술 특허 및 독점 기술

알고리즘의 우위나 법적으로 보호받는 기술적 진입 장벽



감정적 애착 (Emotional Moat)

탁월한 UX와 브랜드 경험을 통해 형성된 고객의 충성도

규모의 경제 (Economies of Scale)

인프라 효율화나 비용 절감을 통해 후발 주자가 따라올 수 없는 가격 우위

데이터 우위 (Data Advantage)

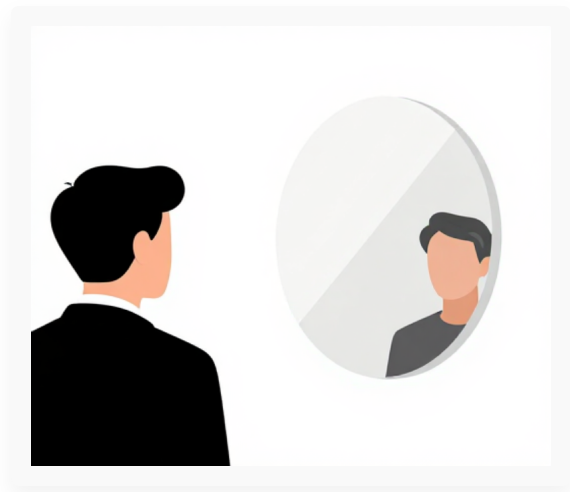
축적된 사용자 데이터를 활용한 고도화된 개인화 및 예측 서비스

"PS는 단순히 벽을 쌓는 사람이 아니라, 성을 영원히 지탱할 해자를 파는 설계자입니다."

- 기능 명세서에 있는 것을 그대로 코딩하는 데 그치지 말고, 이 기능이 어떻게 제품의 '해자'가 될 수 있을지 질문하세요.
- 경쟁자가 코드를 복사할 수는 있어도, 우리가 쌓아올린 네트워크와 데이터의 장벽은 복사할 수 없습니다.

질문 #9 & #10: 파괴적 혁신자의 시선으로 바라보기

외부의 위협과 미래의 변화를 선제적으로 감지하여 제품의 생존 가능성 진단하기



"내부자가 아닌, 파괴자의 시선으로 거울을 보십시오"

QUESTION #9

유능한 창업자가 우리 업계를 뒤흔든다면 뭐라고 말할까요?

- 혁신을 멈춘 PS는 곧 '수구 세력'이 됩니다.
- 우리 제품의 가장 취약한 지점을 외부자의 눈으로 비판해 보세요.

QUESTION #10

어떤 상황에서 우리 제품이 완전히 쓸모없어질까요?

- 기술의 느린 진화(AI, 재생 에너지)와 규제 변화를 주시하세요.
- 거대한 팬데믹뿐 아니라 시장의 체질 변화가 곧 위협입니다.

FUTURE-PROOFING

"전략의 본질은 무엇을 하지 않을지, 그리고 무엇이 우리를 무너뜨릴지 아는 것입니다."

제3장: 최적의 해결책 찾기

PS의 전문성을 활용한 효율적 솔루션 설계 단계

이분법적 사고에서의 탈피

"PM은 문제를 정의하고, PS는 해결만 한다"

이런 낡은 격언은 PS의 잠재력을 제한합니다.



진정한 Problem Solver로서의 PS

- 단순 구현을 넘어 실현 가능성(Feasibility)을 검토합니다.
- 비즈니스 가치와 엔지니어링 비용의 균형을 찾습니다.
- 가장 적은 자원으로 최대의 효과를 내는 지점을 설계합니다.



"좋은 솔루션은 선호도, 수익성, 실현 가능성의 교집합에 있습니다."

이제부터 다음 7가지 질문은 여러분이 '어떻게 만들 것인가'를 넘어 '무엇을 위해 최적화할 것인가'를 고민하게 할 것입니다.

질문 #11 & #12: 의지와 실현 가능성

자원의 제약을 인정하고 DVF 프레임워크로 최적의 해결책 찾기

Q11. 해결 의지가 얼마나 강한가?

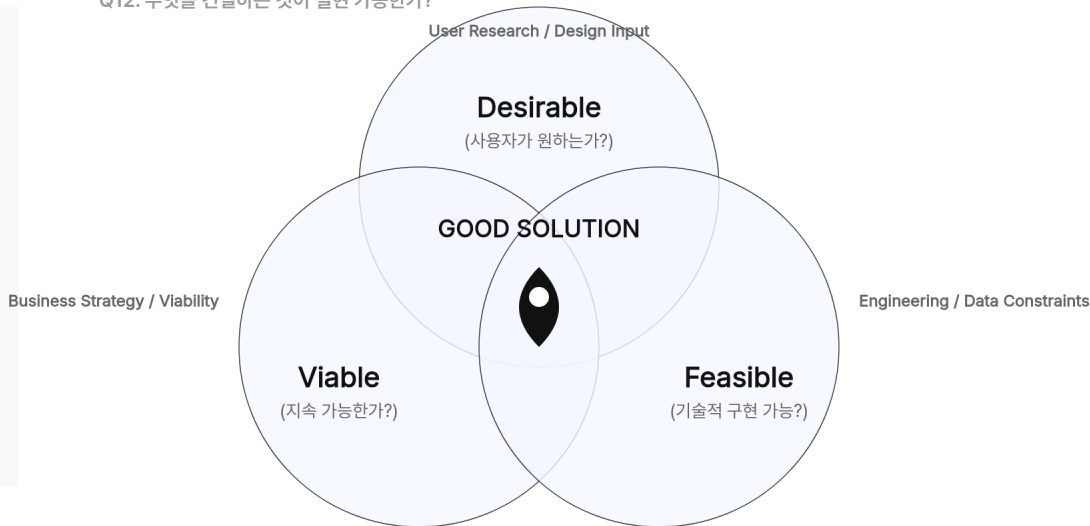
- 얼마나 많은 자원을 할당할 것인가?
- PS가 투입할 수 있는 최대 시간은?

파킨슨 법칙 (Parkinson's Law)

"작업은 주어진 시간과 공간을 채우는 만큼 무한히 확장됩니다."

초기 업무 범위(Scope)를 명확히 하세요.

Q12. 무엇을 건설하는 것이 실현 가능한가?



"해결책은 '만들 수 있는 것'이 아니라 '문제를 해결하는 것'이어야 합니다."

- PS는 기술적 실현 가능성(Feasibility)을 넘어 사업적 생존성(Viability)과 고객 가치(Desirability)를 동시에 고민해야 합니다.

해결책 설계: PS의 '실현 가능성' 검토

단순 배너부터 AI 추천까지, 구현 비용 대비 비즈니스 가치를 판단하는 기술적 리더십

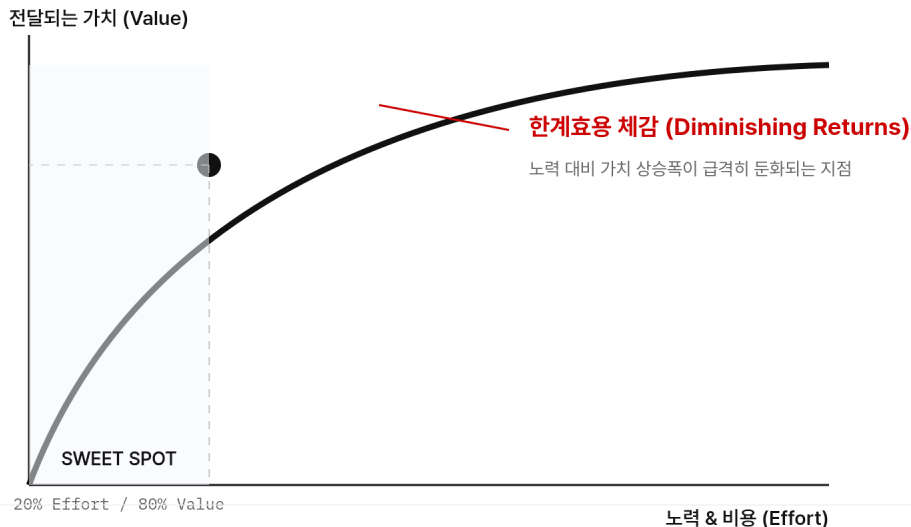


"해결책은 '만들 수 있는 것'이 아니라 '지금 해야 하는 것'의 집합입니다."

- PS는 단순 구현자가 아닙니다. 다양한 옵션 사이의 기술적 비용(Cost)과 복잡도를 투명하게 제안해야 합니다.
- 비즈니스 가치가 비슷하다면, 가장 단순하고 빠른 해결책(Feasible)을 먼저 제안하는 것이 최고의 프로덕트 매너십입니다.

질문 #13: 과잉 엔지니어링의 함정

한계효용 체감의 법칙: 80%의 가치를 만드는 20%의 노력에 집중하기



PS를 위한 Action Plan

1. 복잡성은 곧 비용입니다

가장 복잡한 솔루션이 항상 최선의 해결책은 아닙니다.
유지보수 비용과 개발 기간을 반드시 고려하세요.

2. 80%의 가치에 집중하세요

전체 일정의 절반만으로도 핵심 가치의 80%를
제공할 수 있는 "효율적인 경로"가 있는지 확인하세요.

3. MVP를 통한 반복(Iteration)

"완벽한 솔루션을 위해 몇 달을 소모하는 것은 가장 위험한 투자입니다."

- 고객의 실제 문제를 해결하는 최소 단위(MVP)를 먼저 배포하고 반응을 측정하세요.
- PS의 자부심은 코드의 복잡성이 아니라, 고객 가치의 전달 속도에서 나와야 합니다.

질문 #14: 레드팀 사고 — 스스로의 가장 강력한 비판자가 되기

실제 경쟁자가 공격하기 전에, 우리가 먼저 약점을 찾아내고 보완하는 과정

BLUE TEAM: OUR SOLUTION

"이 기능은 완벽하게 작동하며 고객에게 가치를 줍니다."

- 기능적 요구사항(Functional Req) 충족
- 긍정적인 사용자 시나리오 기반 설계
- PS로서의 자부심과 확신



RED TEAM: THE ADVERSARY

"이 솔루션을 무너뜨릴 가장 치명적인 방법은 무엇인가?"

- "데이터 100배 증가 시 시스템이 마비된다면?"
- "뉴스에서 '개인정보 유출'이라고 보도한다면?"
- "경쟁사가 우리보다 2배 싼 가격에 출시한다면?"



PS를 위한 레드팀 체크리스트

1. 기술적 취약점: 시스템이 특정 옛지 케이스에서 크래시되거나 데이터가 오염될 가능성은?
2. 도덕적/법적 리스크: 의도치 않게 사용자 프라이버시를 침해하거나 차별적인 결과를 낳지는 않는가?
3. 비즈니스 방어력: 경쟁사가 우리의 로직을 그대로 복제했을 때, 우리가 가질 수 있는 독보적인 해자는?

질문 #15: 가장 위험한 가정 식별하기

프로젝트를 무너뜨릴 수 있는 '알 수 없는 위험'을 먼저 검증하는 법

무엇이 '가정'인가요?

- 고객이 이 기능을 진짜로 원할 것이다
- 이 라이브러리가 대용량 트래픽을 견딜 것이다
- API 응답 속도가 100ms 이내일 것이다

"우리는 이미 증명된 사실이 아니라,
수많은 가정 위에 코드를 쌓고 있습니다."



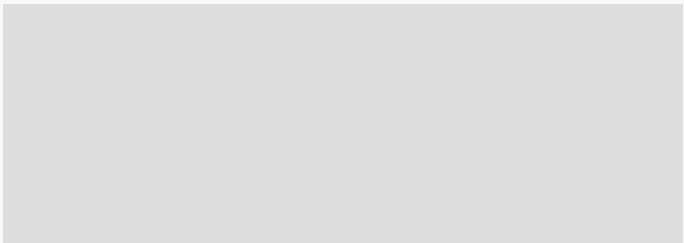
PS를 위한 전략: "코드를 짜기 전에 실험부터 하세요."

전체 아키텍처를 설계하기 전에, 가장 불확실한 기술적 가정을 검증할 수 있는 '최소한의 스파이크(Spike) 코드'나 'PoC'를 먼저 실행하세요.

질문 #16: 가장 작은 가치 단위로 쪼개기

완벽한 100%보다 지금 당장 필요한 60%에 집중하는 사고방식

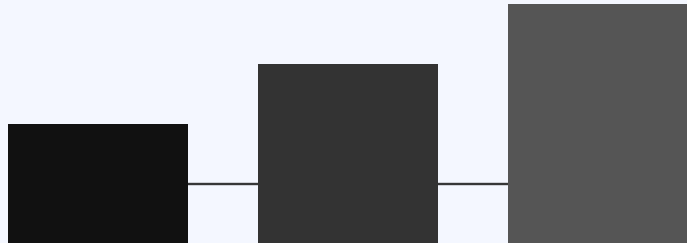
TRADITIONAL : BIG BANG



1년 뒤의 100% 출시

위험: 개발 기간 동안 시장 트렌드와 고객 요구가 변함

PROBLEM SOLVER : INCREMENTAL



Step 1 (60%)

핵심 기능 즉시 전달

Step 2 (80%)

피드백 기반 개선

Step 3 (100%)

완성도 확보

"연구와 출시 사이의 시차는 곧 가치 하락으로 이어집니다."

- PS는 기능을 완성하는 사람이 아니라, 고객의 문제를 빠르게 해결하는 사람입니다.
- 가장 작은 단위(MVP)로 가치를 쪼개어 단계별 빌드 계획을 수립하세요.

질문 #17: 제가 이 문제를 해결하기에 가장 적합한 사람일까요?

성장을 위한 '레고 블록' 나누기: 위임을 통한 영향력 확대



"혼자서 모든 블록을 쥐고 있으면, 당신이 만들 수 있는 것은 당신의 키를 넘지 못합니다."

이해했다면, 이제 놓아줄 때입니다

16개의 질문으로 문제를 완벽히 정의했다면, 그 솔루션을 실행할 기회를 동료에게 양보하세요. 이것은 포기가 아니라 임파워먼트입니다.

팀과 나의 동반 성장 (Multiplier Effect)

주니어 팀원에게는 도전의 기회를, 당신에게는 위임과 멘토링이라는 새로운 리더십 기술을 익힐 수 있는 소중한 자산이 됩니다.

더 큰 문제를 해결할 시간을 확보하세요

반복되는 구현 작업에서 벗어나 더 복잡한 비즈니스 전략과 가치 극대화에 집중할 때, 당신의 영향력은 기술적으로 증명됩니다.

"진정한 Problem Solver는 문제를 해결하는 사람이 아니라,"

"문제가 해결되는 시스템과 팀을 만드는 사람입니다."

요약: 사고력을 날카롭게 하는 17가지 질문 체크리스트

문제를 코드로 풀기 전, '풀 가치가 있는가'를 먼저 검증하세요.

제1장. 문제의 가치 평가

01. 증상인가, 근본 원인인가?
02. 고객에게 미치는 영향력은?
03. 해결 시 회사의 이점은?
04. 장기 전략과 일치하는가?
05. 포기해야 할 기회비용은?
06. 방치 시 발생하는 지연 비용은?

제2장. 중요한 문제 찾기

07. 고객의 진짜 '할 일(JTBD)'은?
08. 강력한 경쟁 우위(해자)가 있는가?
09. 파괴적 혁신가의 시선은 어떤가?
10. 미래에 제품이 쓸모없어질 상황은?

SYSTEM_LOG // ANALYSIS

"PS는 단순히 기능을 구현하는 사람이 아닙니다. 이 질문들은 당신의 코드가 비즈니스 임팩트로 이어지게 만드는 필터입니다."

제3장. 최적의 해결책 찾기

11. 해결하고자 하는 의지가 강한가?
12. 기술/사업적 실현 가능성은?
13. 한계효용 체감이 시작되는 지점은?
14. 레드팀(비판자)의 예상 공격은?
15. 지금 가장 위험한 가정은 무엇인가?
16. 가장 작은 가치 단위(MVP)는?
17. 내가 이 문제를 풀 적임자인가?

TIP: 모든 질문에 답할 필요는 없습니다. 상황에 맞는 핵심 질문을 골라 팀과 논의를 시작하세요.

마치며: 가치를 창조하는 PS의 여정

기술을 넘어 가치로 문제를 해결하는 Problem Solver의 시작

VALUE-DRIVEN ENGINEER

"사고의 근육"을 단련하십시오

PS에서 문제 해결사로의 전환은 한 번의 학습이 아닌 매일의 사고 과정을 훈련하는 과정입니다.

코드 이전에 '가치'를 생각하십시오

코드 한 줄을 적기 전, 이 기능이 고객에게 전달할 '왜'와 '가치'의 단위를 먼저 정의하세요.

기술은 가치를 향한 가장 강력한 엔진입니다

여러분의 전문성은 문제를 해결하는 가장 실질적인 도구입니다.



"최고의 코드는 작성되지 않은 코드이며, 최고의 PS는 비즈니스 가치를 증명하는 사람입니다."

오늘 배운 17가지 질문을 여러분의 백로그와 설계 문서에 통합해 보세요.

당신의 기술로 가치를 증명하세요

엔지니어링은 수단이고,
가치는 목적입니다.

문제에 매료되지 말고, 그 문제가 해결되었을 때
세상이 얻을 가치에 매료되십시오.

당신은 이제 단순한 PS가 아닌, 세상을 바꾸는 '문제 해결사'입니다.